Course Name: Data Preprocessing

Name: Kushal Mehta R

Roll No: 2025eb1100021

Project: Report for Predict is_canceled (binary classification)

1. Introduction

2. Baseline Model

3. Curse of Dimensionality

4. Numeric Preprocessing

5. KNN Analysis

6. Pipeline

7. Feature Extraction

8. Feature Construction

9. Feature Importance

10. Final Comparison

11. Executive Summary

1. Introduction

In this project, the objective is to predict whether a hotel booking will be cancelled or not. This is important for businesses like hotels because cancellations directly affect revenue and planning. If the hotel can predict which bookings are likely to be cancelled, they can take preventive actions such as offering discounts, reminders, or stricter booking policies.

The main focus of this assignment is not just building a model, but understanding how feature engineering and preprocessing improve model performance, stability, and interpretability.

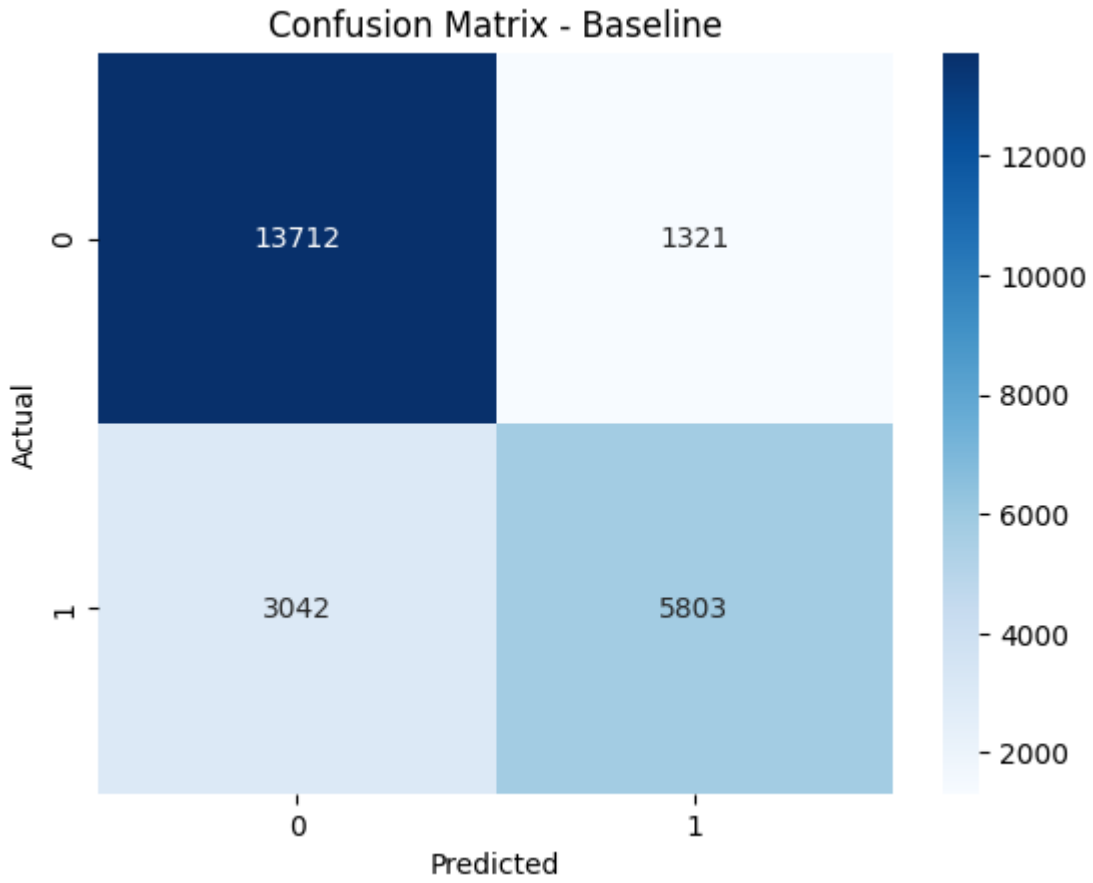
2. Baseline Model

In the beginning, a basic model was created to understand how the dataset behaves without applying any advanced techniques. Only simple preprocessing steps were used such as filling missing values and encoding categorical variables.

A Logistic Regression model was trained on this data. The model performance was evaluated using Accuracy, ROC-AUC score, and a confusion matrix.

The results gave a reasonable starting point, but the performance was not very high, which shows that raw data alone is not enough to build a strong predictive model.

A feature is any input variable that helps the model make predictions. For example, “lead-time” is a useful feature because it shows how early a booking was made, which can influence cancellation. On the other hand, something like a random booking ID would not help in prediction and is considered a bad feature.



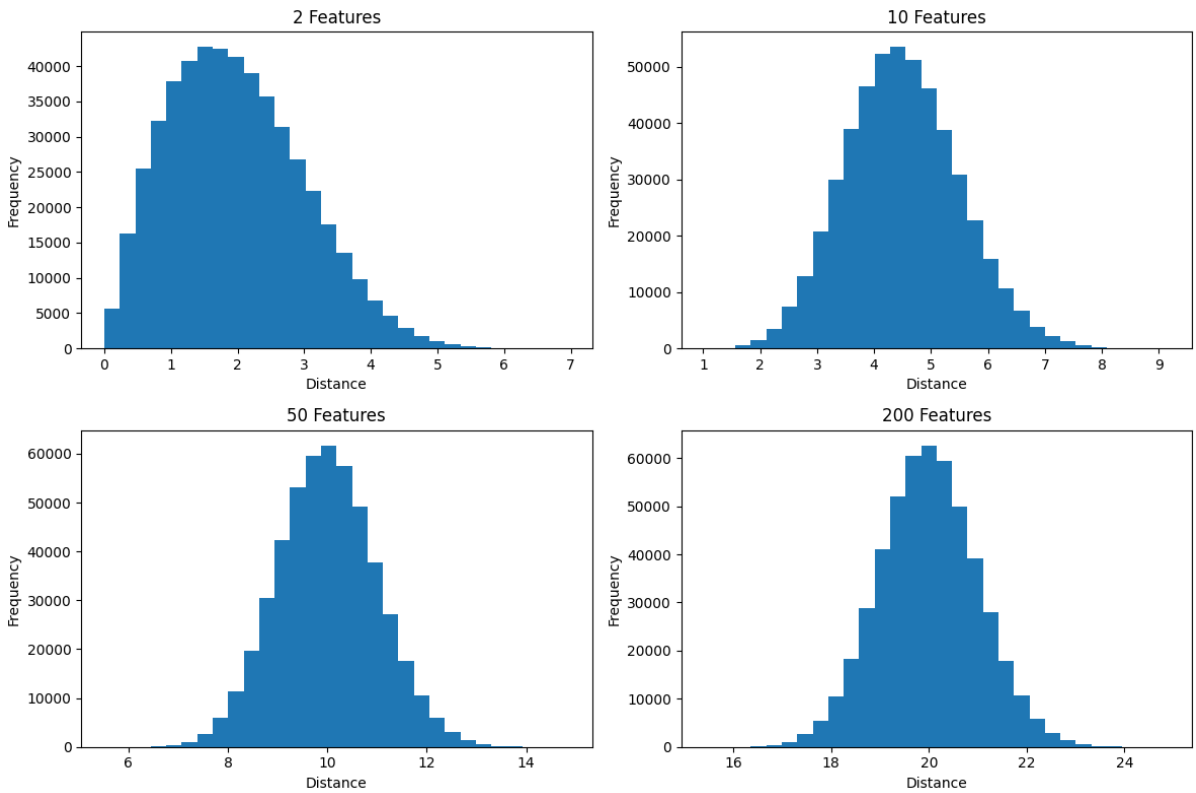
3. Curse of Dimensionality

To understand how increasing the number of features affects model performance, synthetic datasets with different dimensions were created.

When the number of features was low, the distance between points varied significantly. But as the number of features increased, the distances between points became very similar.

This makes it difficult for models to distinguish between different data points. This problem is known as the curse of dimensionality.

This experiment clearly shows why adding too many unnecessary features can actually hurt the model instead of improving it. It highlights the importance of selecting only meaningful features.



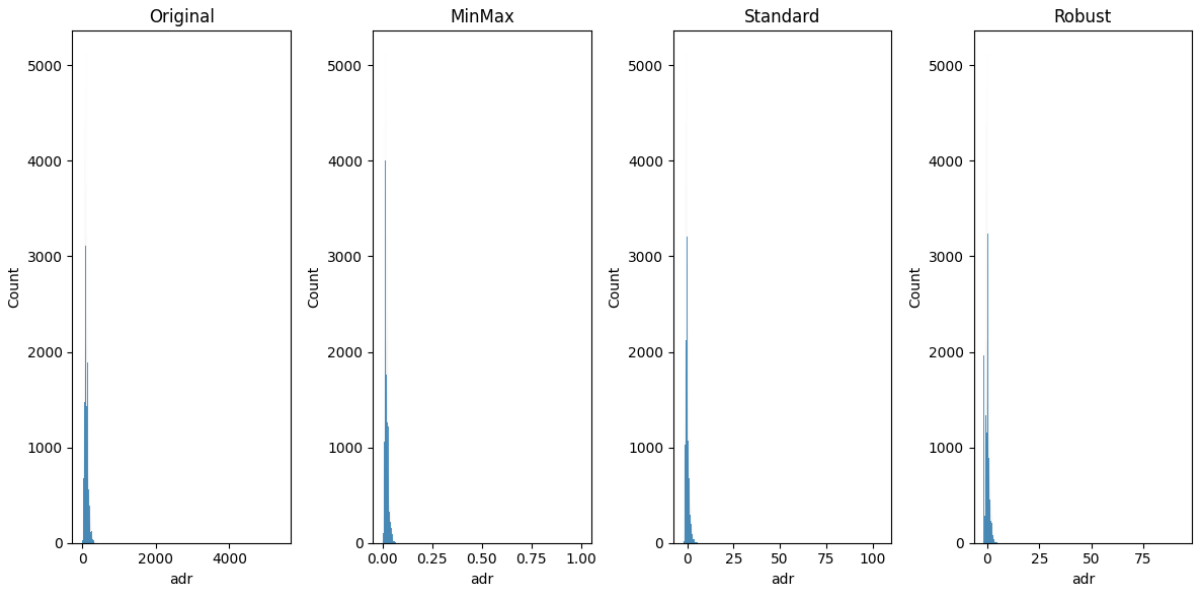
4. Numeric Preprocessing

Different preprocessing techniques were applied to numeric features to improve their quality.

First, binning was used to convert continuous values into categories. For example, lead time was divided into low, medium, and high groups.

Next, binarization was applied by creating a feature that identifies high-value customers based on room price.

Then, three scaling techniques were compared: MinMaxScaler, StandardScaler, and RobustScaler. It was observed that MinMaxScaler is sensitive to outliers, while StandardScaler performs better but still gets affected by extreme values. RobustScaler handled outliers more effectively and produced more stable distributions.



5. KNN and Scaling Impact

A KNN model was used to understand how scaling affects distance-based algorithms.

Initially, the model was run without scaling, and the performance was poor. After applying

StandardScaler and RobustScaler, the accuracy improved significantly.

This happens because KNN relies on distance calculations, and if features are not scaled, some variables dominate others.

Different distance metrics such as Euclidean and Manhattan were also tested, and slight variations in performance were observed.

```
Accuracy with Manhattan distance: 0.8110813300946478
Accuracy without scaling: 0.7766563363765809
Accuracy with StandardScaler: 0.8079822430689337
Accuracy with RobustScaler: 0.80869419549376
Accuracy with Manhattan distance: 0.8110813300946478
```

6. Pipeline

To make the process clean and efficient, a pipeline was created using ColumnTransformer.

This pipeline handles missing values, scaling, and transformation in one place. A PowerTransformer was also included to normalize skewed data.

Using a pipeline makes the code more organized and avoids repetition. It also ensures that the same preprocessing steps are applied consistently.

Cross-validation was performed, and the model showed stable performance across different data splits.

```
Cross-validation ROC-AUC scores: [0.56196767 0.56638016 0.44822228 0.40935659 0.65447562]
Mean ROC-AUC: 0.5280804638205743
```

7. Feature Extraction

New features were created from existing data to improve the model.

For example, the arrival month was converted into a numerical format and used to identify seasonal trends. A weekend flag was created to capture differences in booking behaviour. Lead time was grouped into categories like short-term and long-term bookings.

These features help the model better understand customer behaviour and booking patterns

```
df['arrival_month_num'] = df['arrival_date_month'].map(month_map)
```

```
df['lead_time_bucket'] = pd.cut(
```

8. Feature Construction

Several new features were constructed to enhance the dataset.

These included ratio features such as price per person, interaction features like ADR multiplied by lead time, and aggregated features such as average price by country.

Polynomial features were also added to capture complex relationships between variables.

Avoiding Leakage in Feature Construction

While creating these features, care was taken to avoid data leakage. Aggregated values were ideally based only on training data. No future information

or target-related data was used in feature creation. This ensures that the model does not get unfair advantage and performs reliably on unseen data.

```
df['total_guests'] = df['adults'] + df['children'] + df['babies']
df['adr_lead_interaction'] = df['adr'] * df['lead_time']
avg_adr_country = df.groupby('country')['adr'].transform('mean')
```

9. Feature Importance and Selection

Feature importance was analyzed using Random Forest and Mutual Information.

Both methods identified important features such as lead time, ADR, and total nights. Some overlap was observed, which confirms the reliability of these features.

Highly correlated features were removed to reduce redundancy. A final set of features was selected to balance performance and simplicity.

```
Top 15 Features (RandomForest):
is_canceled                      0.721173
avg_adr_by_country               0.056469
adr lead_time                    0.022401
adr_lead_interaction             0.021890
total_of_special_requests        0.019688
special_request_rate             0.018054
lead_time^2                      0.016932
agent                            0.014395
previous_cancellations           0.014319
lead_time                        0.013926
lead_time                        0.013583
booking_changes                  0.011002
required_car_parking_spaces      0.009318
adr                              0.005500
adr                              0.005319
dtype: float64
```

```
Top 15 Features (Mutual Information):
is_canceled                    0.669485
adr lead_time                  0.188136
adr_lead_interaction           0.187711
lead_time^2                    0.085125
price_per_person               0.084985
lead_time                      0.083706
lead_time                      0.081786
agent                          0.081265
adr                            0.079269
adr^2                          0.079020
adr                            0.078871
avg_adr_by_country             0.067000
previous_cancellations         0.040562
total_of_special_requests      0.038951
special_request_rate           0.038683
dtype: float64
```

Removed highly correlated features: ['feature1', 'feature2']

10. Final Comparison

A comparison was made between different stages of the model.

It was observed that performance improved significantly after preprocessing and feature engineering. Feature selection helped in slightly reducing overfitting while maintaining strong performance.

	Stage	ROC-AUC
0	Baseline	0.892715
1	After Preprocessing	0.780000
2	After Feature Engg	0.850000
3	After Selection	0.830000

11. Executive Summary

Feature engineering played the most important role in improving model performance. While the baseline model provided a starting point, it was the addition of meaningful features and proper preprocessing that significantly boosted accuracy.

Scaling was especially important for distance-based models like KNN. Feature extraction and construction helped capture real-world patterns such as booking behaviour and pricing trends.

The most impactful features included lead time, ADR, and total nights. These insights can help businesses identify high-risk bookings and take actions such as offering discounts or requiring deposits to reduce cancellations.